第2章 线性表

提纲

CONTENTS

2.1 线性表的定义

2.2 线性表的顺序存储结构

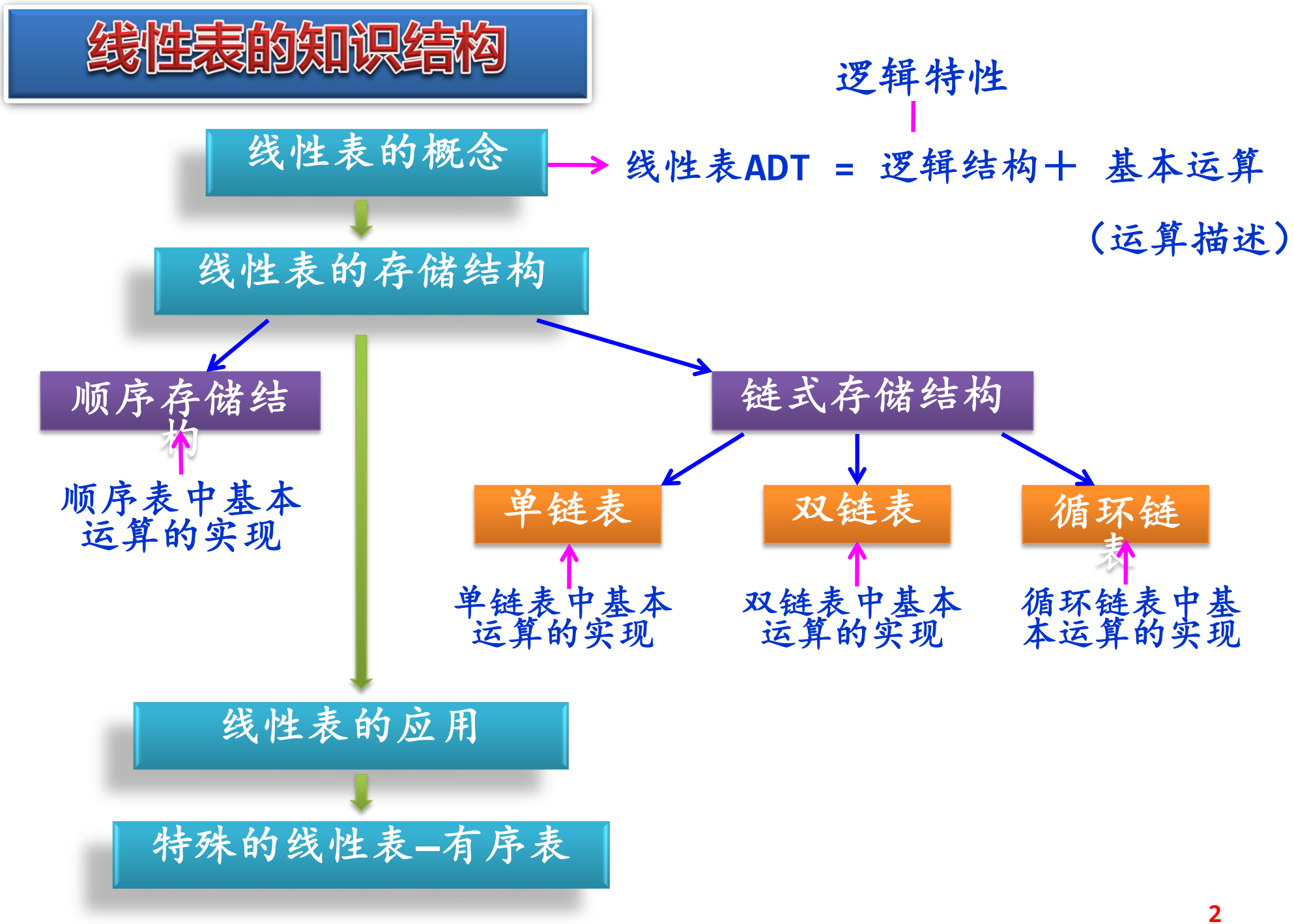
2.3 线性表的链式存储结构

2.4 顺序表和链表的比较

2.5 线性表的应用

2.6 STL中的线性表

线性表的知识结构


逻辑特性
线性表的概念
线性表ADT = 逻辑结构＋ 基本运算
（运算描述）
线性表的存储结构
顺序存储结构
顺序表中基本运算的实现
链式存储结构
单链表
双链表
循环链表
单链表中基本运算的实现
双链表中基本运算的实现
循环链表中基本运算的实现
线性表的应用
特殊的线性表—有序表

2.1 线性表的定义

2.1.1 什么是线性表

例1：字母有限序列（A,D,E,C,F,I)。需要对该序列的操作包括：插入、删除、判空等。

例2：数字有限序列（33,78,101,99,24,75)。需要对该序列的操作包括：插入、删除、判空等。

例3：对象有限序列：假设用class定义学生类Student，类Student的个体组成有限序列（zhang，wang，liu，li）需要对该序列的操作包括：插入、删除、判空等。

2.1 线性表的定义

2.1.1 什么是线性表

线性表是具有相同特性的数据元素的一个有限序列。

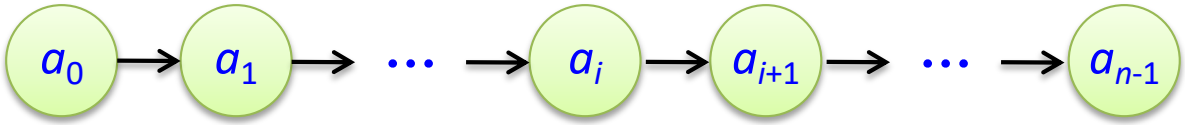
- 所有数据元素类型相同。
- 线性表是有限个数据元素构成的。
- 线性表中数据元素与位置相关，即每个数据元素有唯一的序号。

线性表中所含元素的个数叫做线性表的长度，用n表示，$n \geq 0$。$n=0$时，表示线性表是一个空表，即表中不包含任何元素。

线性表的逻辑结构表示

$$(a_0, a_1, \cdots, a_i, a_{i+1}, \cdots, a_{n-1})$$

用图形表示的逻辑结构：



说明

线性表中每个元素a_i的唯一位置通过序号或者索引i表示，为了算法设计方便，将逻辑序号和存储序号统一，均假设从0开始，这样含n个元素的线性表的元素序号i满足$0 \leqslant i \leqslant n-1$。

2.1.2 线性表的抽象数据类型描述

ADT List

{

数据对象：

$D=\{a_i \mid 0 \leqslant i \leqslant n-1,\ n \geqslant 0\}$

数据关系：

$r=\{<a_i, a_{i+1}> \mid a_i, a_{i+1} \in D,\ i=0, \cdots, n-2\}$

基本运算：

CreateList(a)：由整数数组a中的全部元素建立线性表的相应存储结构。

Add(e)：将元素e添加到线性表末尾。

getlength()：求线性表的长度。

GetElem(int i)：求线性表中序号为i的元素。

SetElem(int i, T e)：设置线性表中序号i的元素值为e。

GetNo(T e)：求线性表中第一个值为e的元素的序号。

Insert(int i, T e)：在线性表中插入数据元素e作为第i个元素。

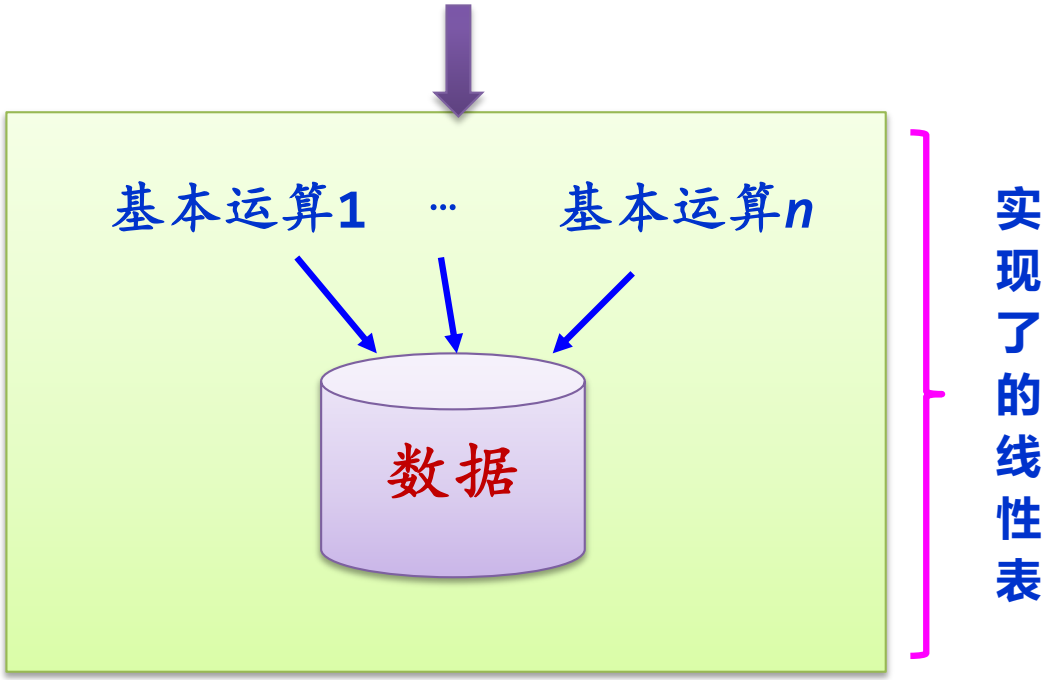
Delete(int i)：在线性表中删除第i个数据元素。

DispList()：输出线性表的所有元素。

}

线性表的作用

在实现线性表基本算法的前提下，
可以用它来解决具体问题



- 程序员可以直接使用它来存放数据——作为存放数据的容器。
- 程序员可以直接使用它的基本运算——完成更复杂的功能。

2.2 线性表的顺序存储结构

2.2.1 线性表的顺序存储结构—顺序表

线性表的顺序存储结构：把线性表中的所有元素按照顺序存储方法进行存储。



按逻辑顺序依次存储到存储器中**一片连续的存储空间**中。

长度为n的线性表存放在顺序表中

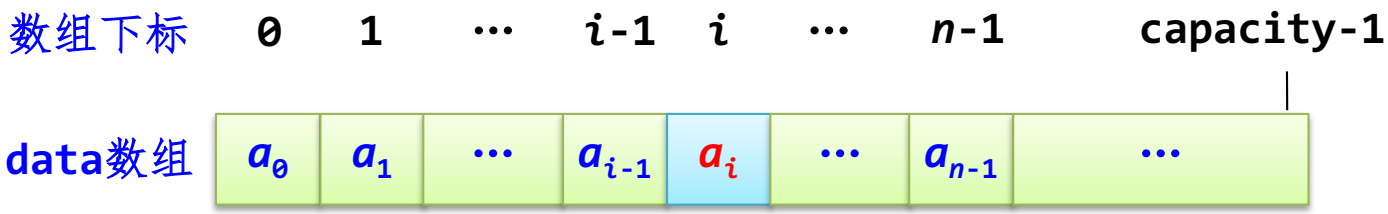
逻辑结构

线性表

$(a_0, a_1, \ldots, a_i, \ldots a_{n-1})$

直接映射

存储结构



- data数组存放线性表元素
- data数组的容量（存放最多的元素个数）为capacity。
- 线性表中实际数据元素个数length

```
const int initcap=5;                    //顺序表的初始容量（5）
template <typename T>
class SqList                            //顺序表类模板
{
public:
  T* data;                              //存放顺序表元素空间的指针
  int capacity;                         //顺序表的容量
  int length;                           //存放顺序表的长度
//线性表的基本运算算法
};
```

2.2.2 线性表基本运算算法在顺序表中的实现

在动态分配顺序表的空间时，初始容量设置为initcapacity，当添加或者插入元素可能需要扩大容量，在删除元素时可能需要减少容量。

```
void recap(int newcap)                    //改变顺序表的容量为newcap
{  if (newcap<=0) return;
   T* olddata=data;
   data=new T[newcap];                    //分配新空间
   capacity=newcap;                       //更新容量
   for(int i=0;i<length;i++)              //元素复制
      data[i]=olddata[i];
   delete [] olddata;                     //释放原空间
}
```

1.整体建立顺序表

由含若干个元素的数组*a*的全部元素整体创建顺序表，即依次将*a*中的元素添加到data数组的末尾，当出现上溢出时按实际元素个数length的两倍扩大容量。

```
void CreateList(T a[],int n)          //由数组a中元素整体建立顺序表
{
   for (int i=0;i<n;i++)
   {  if (length==capacity)           //容量不够时
        recap(2*length);              //扩大容量
      data[length]=a[i];
      length++;                       //添加后元素个数增加1
   }
}
```

2.顺序表基本运算算法

（1）顺序表的初始化和销毁

构造函数

```
SqList()                          //构造函数
{   data=new T[initcap];          //为data分配初始容量大小的空间
    capacity=initcap;             //初始化容量
    length=0;                     //初始时置length为0
}
```

初始化复制构造函数（拷贝构造函数）

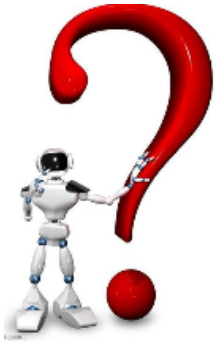
```
SqList(const SqList<T>& s)              //初始化复制构造函数
{  capacity=s.capacity;                 //复制容量
   length=s.length;                     //复制长度
   data=new T[capacity];                //为当前顺序表分配空间
   for (int i=0;i<length;i++)           //元素复制
      data[i]=s->data[i];
}
```

析构函数

```
~SqList()                    //析构函数
{
   delete [] data;           //释放data指向的空间
}
```

（2）将元素e添加的线性表末尾Add(e)

```
void Add(T e)                          //在线性表的末尾添加一个元素e
{  if (length==capacity)               //顺序表空间满时倍增容量
      recap(2*length);
   data[length]=e;                     //添加元素e
   length++;                           //长度增1
}
```



时间复杂度是多少?

（3）求线性表的长度getlength()

```
int Getlength()                    //求顺序表的长度
{
    return length;
}
```

（4）求线性表中序号为i的元素GetElem(i,&e)

```
bool GetElem(int i, T& e)      //求序号i的元素值
{  if (i<0 || i>=length)
       return false;           //参数错误时返回false
   e=data[i];                  //取元素值
   return true;                //成功找到元素时返回true
}
```

（5）设置线性表中序号为i的元素SetElem(i, e)

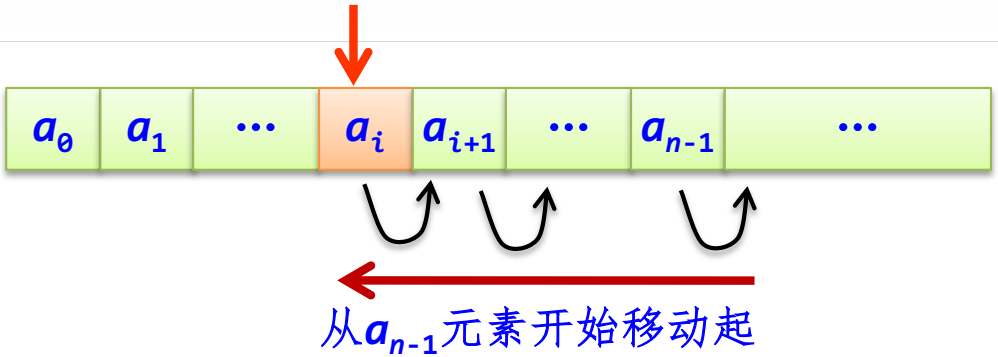
```
bool SetElem(int i,T e)                    //设置序号i的元素值
{  if (i<=0 || i>=length)                  //参数错误时返回false
      return false;
   data[i]=e;
   return true;
}
```

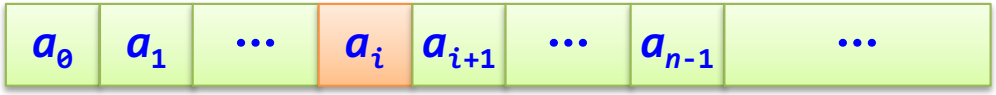
（6）求线性表中第一个值为e的元素的逻辑序号GetNo(e)

```
int GetNo(T e)                          //查找第一个为e的元素的序号
{  int i=0;
   while(i<length && data[i]!=e)
      i++;                              //查找元素e
   if (i>=length)                       //未找到时返回-1
      return -1;
   else
      return i;                         //找到后返回其序号
}
```

(7) 在线性表中插入e作为第i个元素Insert(i, e)

```
bool Insert(int i, T e)                    //在线性表中序号i位置插入元素e
{  if (i<0 || i>length)                    //参数i错误返回false
      return false;
   if (length==capacity)                   //满时倍增容量
      recap(2*length);
   for (int j=length;j>i;j--)              //data[i]及后元素后移一个位置
      data[j]=data[j-1];
   data[i]=e;                              //插入元素e
   length++;                               //长度增1
   return true;
}
```





主要时间花在元素移动上。有效插入位置**i**的取值是**0**～**n**，共有**n+1**个位置可以插入元素：

- 当**i=0**时，移动次数为**n**，达到最大值。
- 当**i=n**时，移动次数为**0**，达到最小值。
- 其他情况，需要移动**data[i..n-1]**的元素，移动次数为**(n-1)-i+1=n-i**。

$p_i = \frac{1}{n+1}$ 所需移动元素的平均次数为：

$$\sum_{i=0}^{n} p_i (n-i) = \frac{1}{n+1}\sum_{i=0}^{n}(n-i) = \frac{1}{n+1} \times \frac{n(n+1)}{2} = \frac{n}{2}$$

插入算法的平均时间复杂度为O(n)。

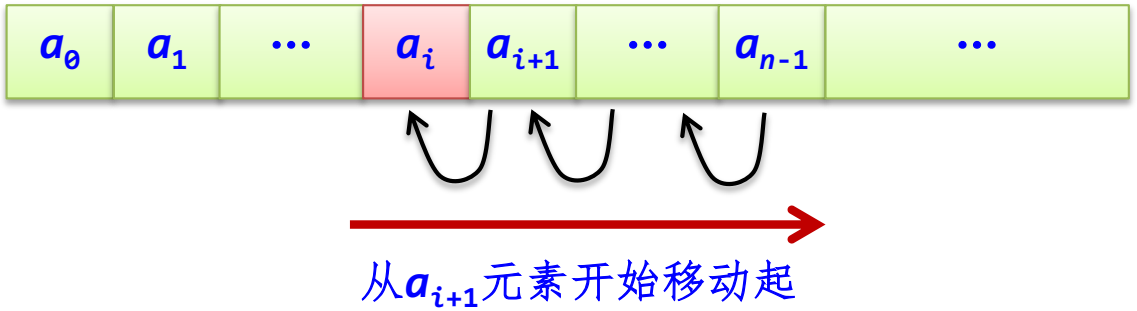
说明

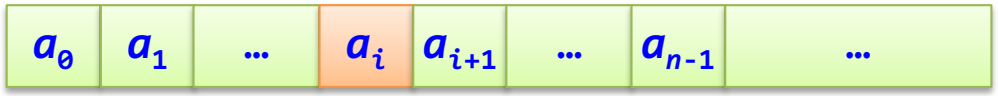
扩容运算recap()在n次插入中仅仅调用一次，其平摊时间为O(1)，上述算法时间分析中可以忽略它。

（8）在线性表中删除第i个数据元素Delete(i)

```
bool Delete(int i)                          //在线性表中删除序号i的元素
{  if (i<0 || i>=length)                    //参数i错误返回false
      return false;
   for(int j=i;j<length-1;j++)
      data[j]=data[j+1];                    //将data[i]之后元素前移一个位置
   length--;                                //长度减1

   if (capacity>initcap && length<=capacity/4)
      recap(capacity/2);                    //满足缩容条件则容量减半
   return true;
}
```





主要时间花在元素移动上。有效删除位置i的取值是$0 \sim n-1$，共有n个位置可以删除元素：

- 当$i=0$时，移动次数为$n-1$，达到最大值。
- 当$i=n-1$时，移动次数为0，达到最小值。
- 其他情况，需要移动data[$i+1..n-1$]的元素，移动次数为$(n-1)-(i+1)+1=n-i-1$。

$p_i = \frac{1}{n}$ 所需移动元素的平均次数为：

$$\sum_{i=0}^{n-1} p_i(n-i-1) = \frac{1}{n}\sum_{i=0}^{n-1}(n-i-1) = \frac{1}{n} \times \frac{n(n-1)}{2} = \frac{n-1}{2}$$

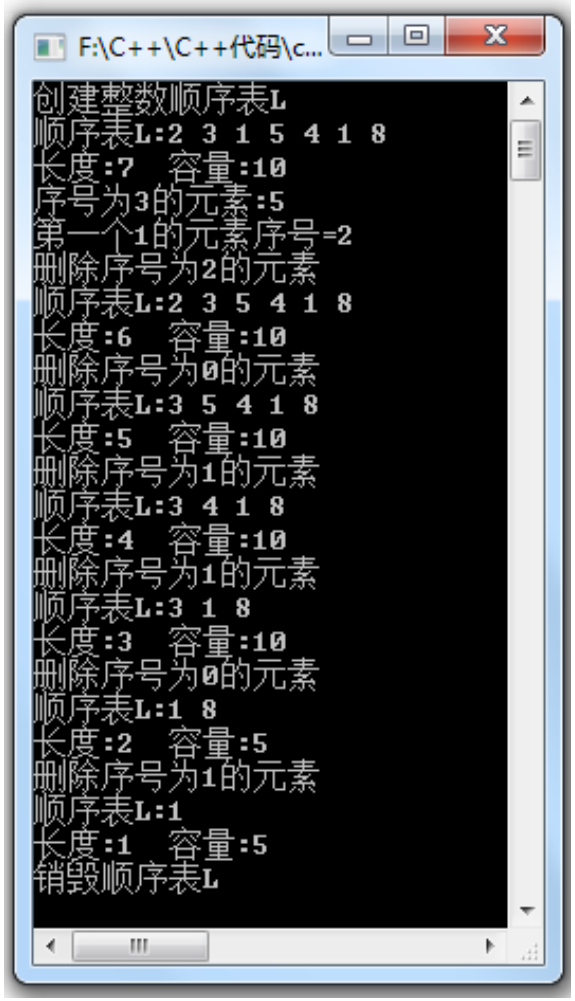
删除算法的平均时间复杂度为$O(n)$。

(9) 输出线性表所有元素DispList()

```
void DispList()                          //输出顺序表L中所有元素
{  for (int i=0;i<length;i++)            //遍历顺序表中各元素值
      cout << data[i] << " ";
   cout << endl;
}
```

程序验证

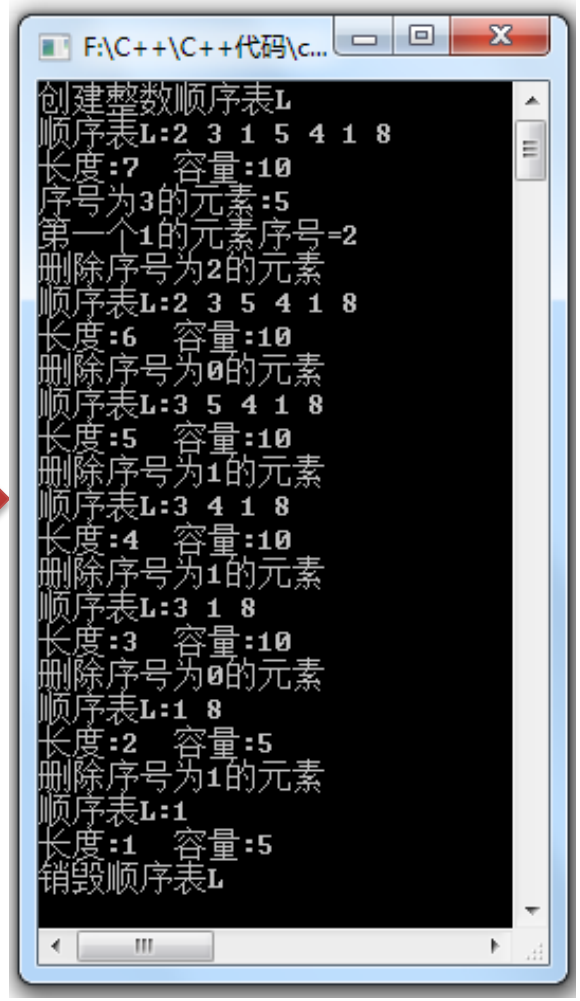
```
#include "SqList.cpp" //引用顺序表类
int main()
{   int i,e;
    SqList<int> L;  //建立类型为int的顺序表对象L
    cout << "创建整数顺序表L" << endl;
    L.Insert(0,2);      //插入元素2
    L.Insert(1,3);      //插入元素3
    L.Insert(2,1);      //插入元素1
    L.Insert(3,5);      //插入元素5
    L.Insert(4,4);      //插入元素4
    L.Insert(5,1);      //插入元素1
    L.Add(8);           //添加整数8
    cout << "顺序表L:"; L.DispList();
    cout << "长度:" << L.length << "  容量:"
         << L.capacity << endl;
    i=3; L.GetElem(i,e);
    cout << "序号为" << i << "的元素:" << e <<
endl;
```



程序验证

```
    e=1;
    cout << "第一个" << e << "的元素序号=" <<
          L.GetNo(e) << "\n";
    i=2; cout << "删除序号为" << i << "的元素\n";
    L.Delete(i);
    cout << "顺序表L:";L.DispList();
    cout << "长度:" << L.length << "  容量:"
          << L.capacity << endl;
    int b[]={0,1,1,0,1};
    for (int i=0;i<5;i++)
    {   cout << "删除序号为" << b[i] << "的元素\n";
        L.Delete(b[i]);
        cout << "顺序表L:";L.DispList();
        cout << "长度:" << L.length << "  容量:"
          << L.capacity << endl;
    }
    cout << "销毁顺序表L" << endl;
    return 0;
}
```





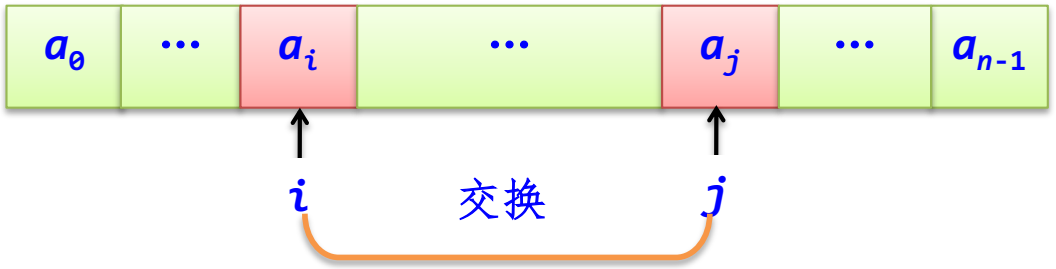
2.2.3 顺序表的应用算法设计示例

1.基于顺序表基本操作的算法设计

【例2.1】对于含有*n*个整数元素的顺序表L，设计一个算法将其中所有元素逆置。

例如L=(1，2，3，4，5)，逆置后L=(5，4，3，2，1)。并给出算法的时间复杂度和空间复杂度。





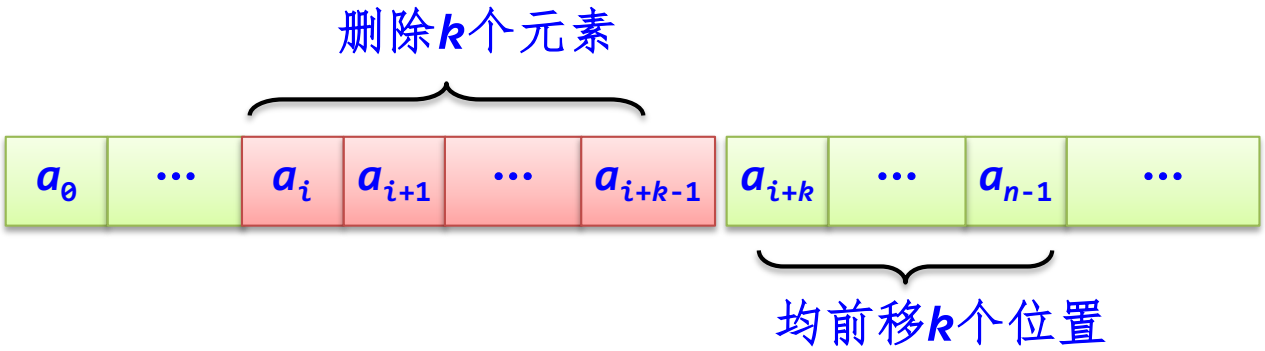

```
void Reverse(SqList<T>& L)                  //求解算法
{  int i=0,j=L.length-1;
   while (i<j)
   {  swap(L.data[i], L.data[j]);    //序号i和j的两个元素交换
      i++; j--;
   }
}
```

【例2.2】假设有一个整数顺序表L，设计一个算法用于删除从序号i开始的k个元素。

例如L=（1，2，3，4，5），删除i=1开始的k=2个元素后L=（1，4，5）。

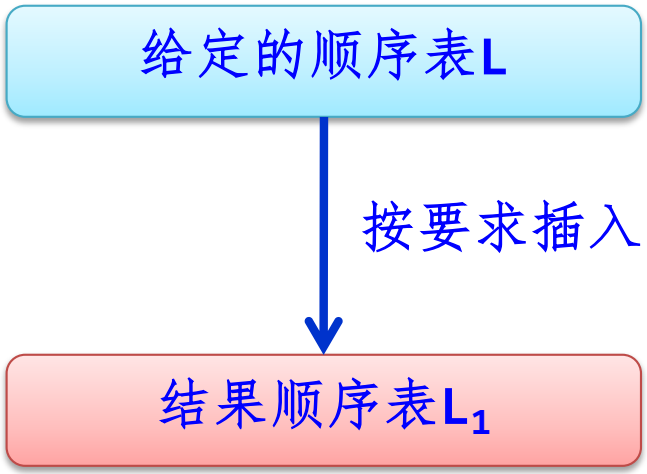


在参数正确时，直接将a_{i+k}～a_{n-1}的所有元素依次前移k个位置。




```
bool Deletek(SqList<T>& L,int i,int k)                    //求解算法
{  if (i<0 || k<1 || i+k<1 || i+k>L.length)
      return false;                          //参数i和k错误返回false
   for (int j=i+k;j<L.length;j++)            //删除k个元素
       L.data[j-k]=L.data[j];
   L.length-=k;                              //长度减k
   return true;
}
```

2.基于整体建立顺序表的算法设计

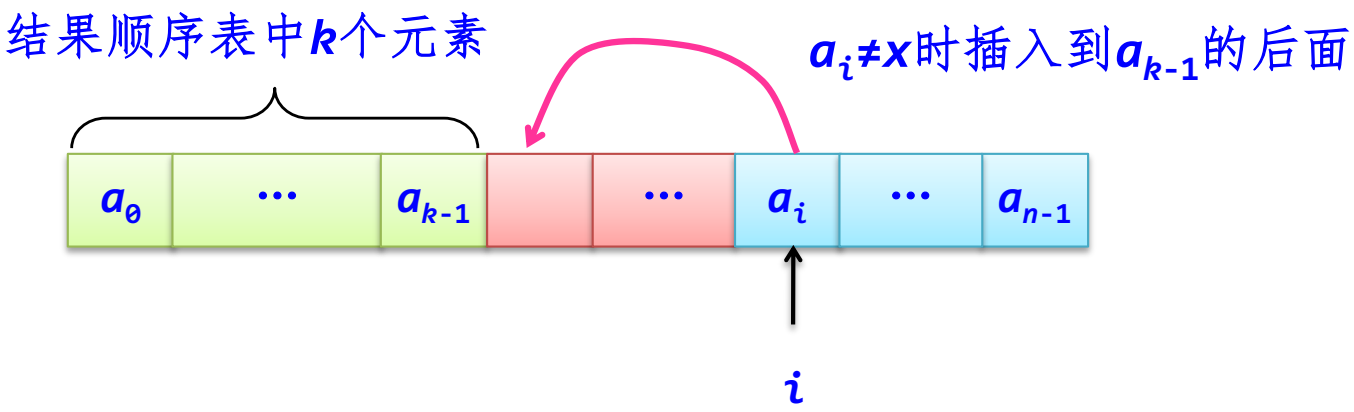


如果两者可以共享，直接在L中操作产生结果顺序表

【例2.3】对于含有n个整数元素的顺序表L，设计一个算法用于删除其中所有值为x的元素。

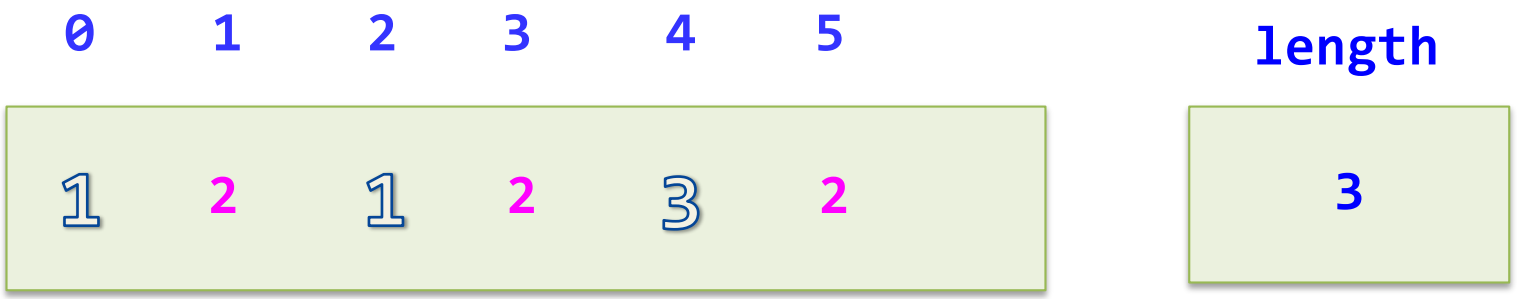
例如L=(1，2，1，5，1)，若x=1，删除后L=(2，5)。并给出算法的时间复杂度和空间复杂度。

解法1：对于整数顺序表L，删除其中所有x元素后得到的结果顺序表可以与原L共享，所以求解问题转化为新建结果顺序表。



删除顺序表中所有值为x的元素（方法1）演示

删除所有x=2的元素（k记录保留的元素个数，初值=0）：

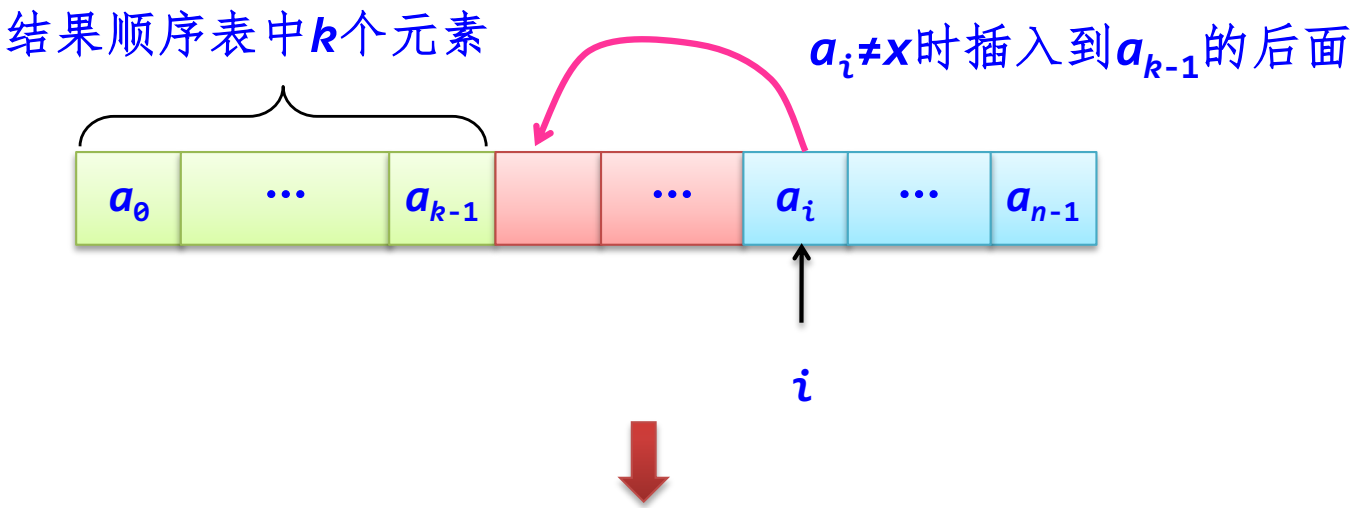


k=3

k=3，L->length=k=3

删除完成

以整体建立顺序表算法为基础！



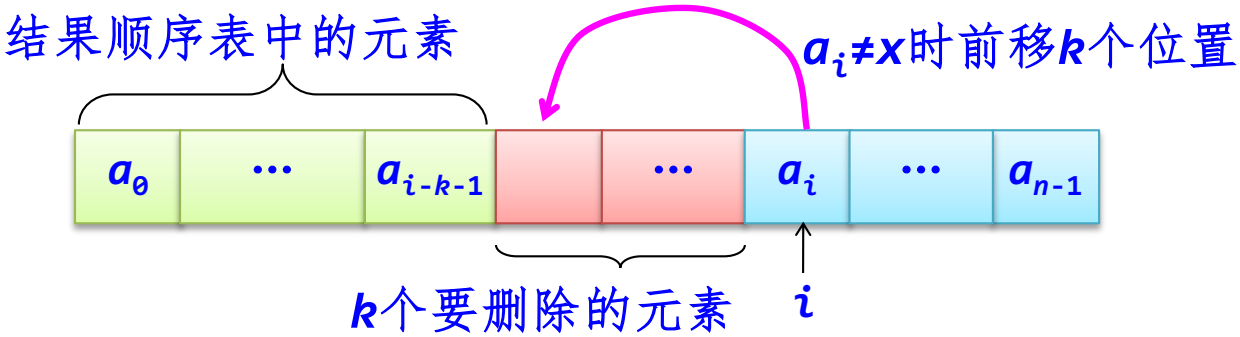

```
template <typename T>
void Deletex1(SqList<T>& L, int x)   //求解算法1
{  int k=0;
   for (int i=0;i<L.length;i++)
      if (L.data[i]!=x)              //将不为x的元素插入到data中
      {  L.data[k]=L.data[i];
         k++;
      }
    L.length=k;                      //重置L的长度为k
}
```

解法2：前移法，对于整数顺序表L，从头开始遍历L，用k累计当前为止值为x的元素个数（初始值为0），处理当前序号为i的元素a_i：

（1）若a_i是不为x的元素，此时前面有k个为x的元素，将a_i前移k个位置，继续处理下一个元素。

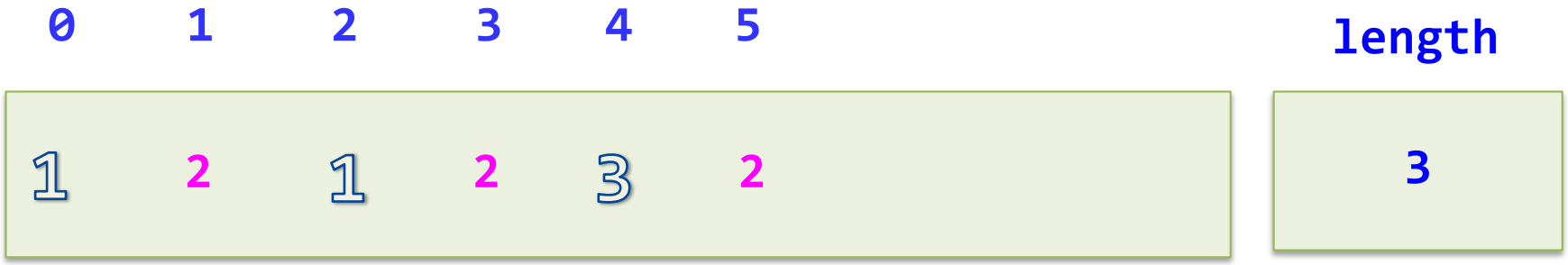
（2）若是为x的元素，置k++，继续处理下一个元素。

最后将L的长度减少k。



删除顺序表中所有值为x的元素（方法2）演示

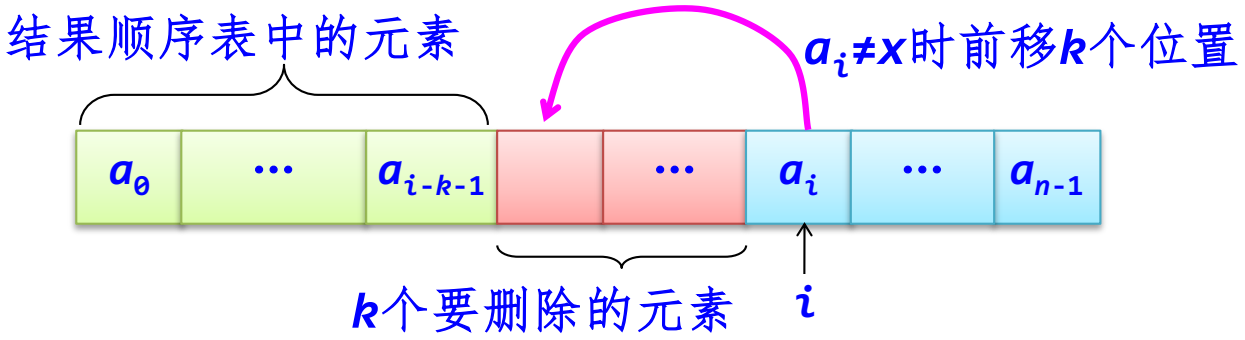
删除所有x=2的元素（k记录删除的元素个数，初值=0）



k=3

顺序表长度=6-k=3

删除完成

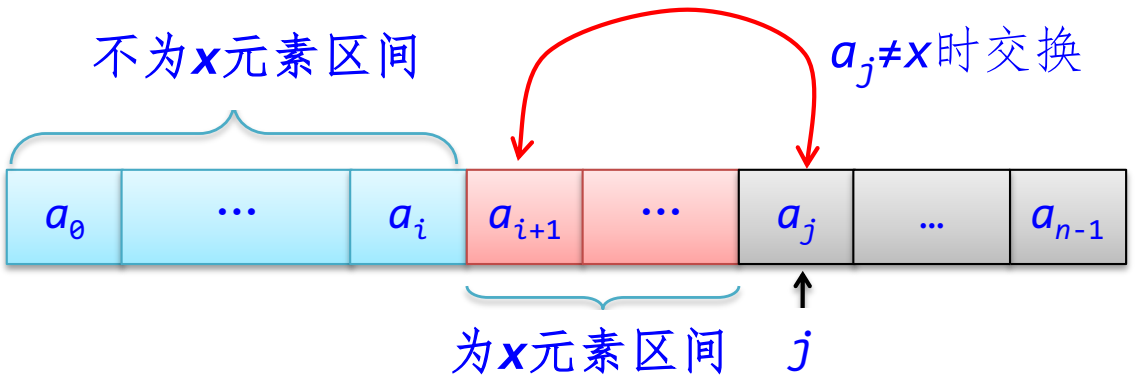



```
template <typename T>
void Deletex2(SqList<T>& L, int x)    //求解算法2
{  int k=0;                           //累计等于x的元素个数
   for (int i=0;i<L.length;i++)
      if (L.data[i]!=x)               //将不为x的元素前移k个位置
         L.data[i-k]=L.data[i];
      else                            //累计删除的元素个数k
         k++;
   L.length-=k;                       //将L的长度减少k
}
```

```
template <typename T>
void Deletex1(SqList<T>& L, int x)   //求解算法1
{  int k=0;
   for (int i=0;i<L.length;i++)
      if (L.data[i]!=x)              //将不为x的元素插入到data中
      {  L.data[k]=L.data[i];
         k++;
      }
   L.length=k;                       //重置L的长度为k
}
```

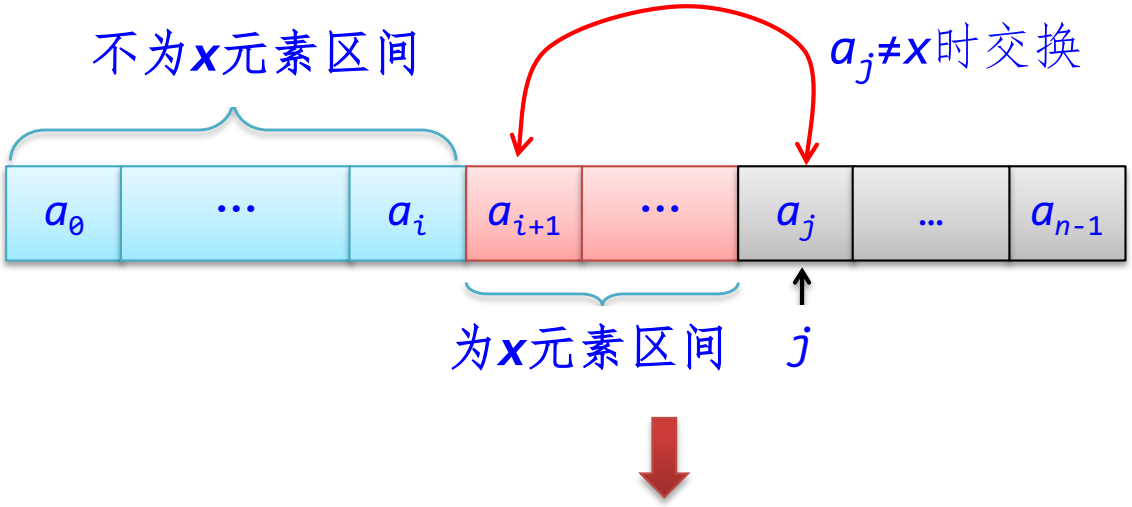
```
template <typename T>
void Deletex2(SqList<T>& L, int x)   //求解算法2
{  int k=0;                          //累计等于x的元素个数
   for (int i=0;i<L.length;i++)
      if (L.data[i]!=x)              //将不为x的元素前移k个位置
         L.data[i-k]=L.data[i];
      else                           //累计删除的元素个数k
         k++;
   L.length-=k;                      //将L的长度减少k
}
```

解法3：由解法2延伸出区间划分法



初始时，“不为x的区间”为空 ⇨ i=-1，j从0开始遍历，“为x的区间”是a[i+1..j-1]

- 若a[j]=x，跳过，j++。
- 若a[j]≠x，操作是，先执行i++，将a[j]与a[i]进行交换，再执行j++继续遍历其余元素。




```
template <typename T>
void Deletex3(SqList<T>& L, int x)          //求解算法3
{  int i=-1,j=0;
   while (j<L.length)                       //j遍历所有元素
   {  if (L.data[j]!=x)                     //找到不为x的元素a[j]
      {  i++;                               //扩大不为x的区间
         if (i!=j)
            swap(L.data[i],L.data[j]);      //序号i和j的两个元素交换
      }
      j++;                                  //继续遍历
   }
   L.length=i+1;                            //将L的长度置为i+1
}
```

扩展

各种顺序表的高效算法设计

- 设计一个算法，从一给定的顺序表L中删除元素值在x到y（$x \leqslant y$）之间的所有元素，要求算法的时间复杂度为$O(n)$，空间复杂度为$O(1)$。
- 设计一个算法从有序顺序表中删除重复的元素，并使剩余元素间的相对次序保持不变。
- …

3.有序顺序表的算法设计

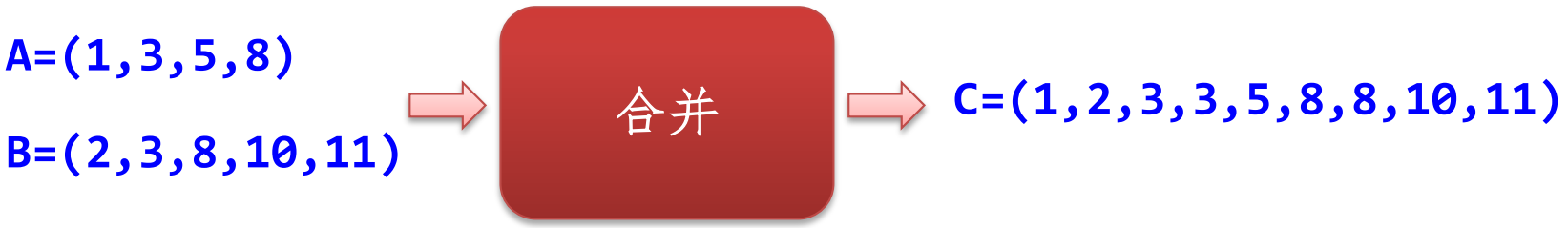
- 有序表是指按元素值或者某属性值递增或者递减排列的线性表，有序表是线性表的一个子集。
- 有序顺序表是有序表的顺序存储结构。
- 对于有序表可以利用其元素的有序性提高相关算法的效率，二路归并就是有序表的一种经典算法。

A=(1,3,8,23,30)

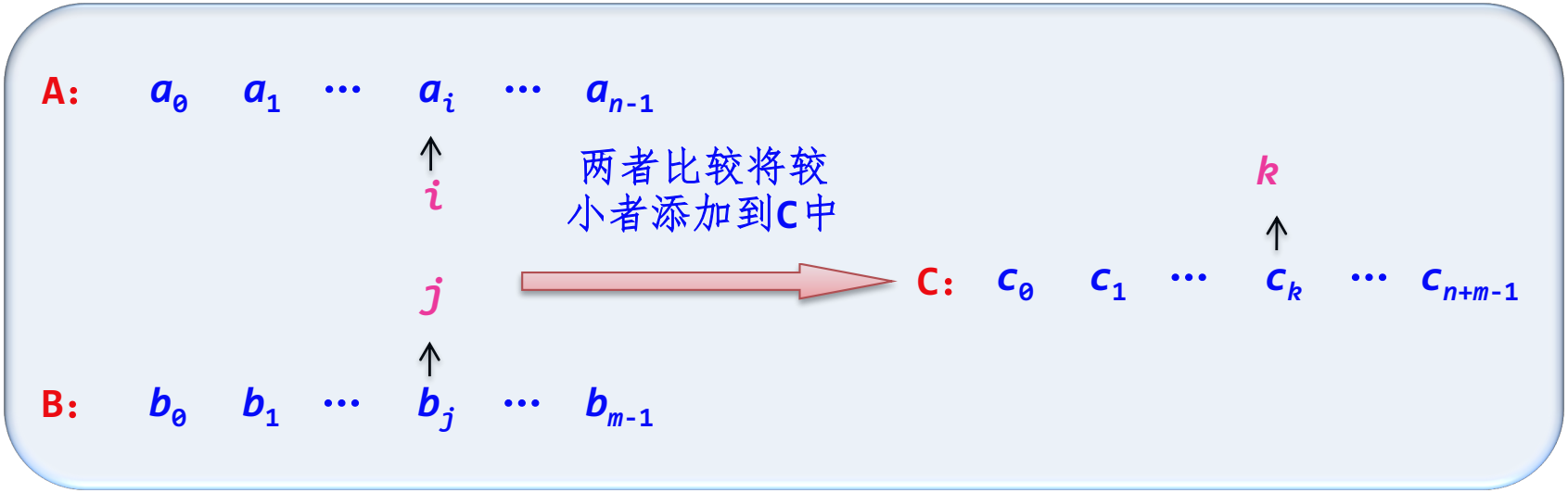


一个递增有序整数顺序表

【例2.4】有两个按元素值递增有序的整数顺序表A和B，设计一个算法将顺序表A和B的全部元素合并到一个递增有序顺序表C中。并给出算法的时间复杂度和空间复杂度。



二路归并：



i遍历A，j遍历B，均从0开始

while i,j都没有超界

　　a_i与b_j比较：较小元素添加到C中，后移相应指针

将没有遍历完的元素添加到C中

```
template <typename T>
void Merge2(SqList<T> A,SqList<T> B,SqList<T>& C)
{  int i=0,j=0;                          //i用于遍历A,j用于遍历B
   while (i<A.length && j<B.length)  //两个表均没有遍历完毕
   {  if (A.data[i]<B.data[j])
      {  C.Add(A.data[i]);               //归并A[i]:将较小的A[i]添加到C中
         i++;
      }
      else                               //归并B[j]:将较小的B[j]添加到C中
      {  C.Add(B.data[j]);
         j++;
      }
   }
```

```
    while (i<A.length)                    //若A没有遍历完毕
    {  C.Add(A.data[i]);                  //归并A中剩余元素
       i++;
    }

    while (j<B.length)                    //若B没有遍历完毕
    {  C.Add(B.data[j]);                  //归并B中剩余元素
       j++;
    }
}
```

- 算法中尽管有多个**while**循环语句，但恰好对顺序表**A**、**B**中每个元素均访问一次，所以时间复杂度为**O(*n*+*m*)** 。
- 算法中需要在临时顺序表**C**中添加***n*+*m***个元素，所以算法的空间复杂度也是**O(*n*+*m*)**。



二路归并中，若两个有序表的长度分别为n、m，算法的主要时间花费在元素比较上，那么比较次数是多少呢？

- 最好的情况下，整个归并中仅仅是较长表的第一个元素与较短表每个元素比较一次，此时元素比较次数为MIN(n，m)（为最少元素比较次数），如A=(1，2，3)，B=(4，5，6，7，8)，只需比较3次。
- 最坏的情况下，这$n+m$个元素均两两比较一次，比较次数为$n+m-1$（为最多元素比较次数），如A=(1，3，5，7)，B=(2，4，6)，需要比较6次。

2009年全国计算机学科考研题

【例2.5】一个长度为L（L≥1）的升序序列S，处在第⌈L/2⌉个位置的数称为S的**中位数**。

例如：若序列S_1=(11，13，15，17，19)，则S1的中位数是15。

两个序列的中位数是含它们所有元素的升序序列的中位数。例如，若S_2=(2，4，6，8，20)，则S_1和S_2的中位数是11。

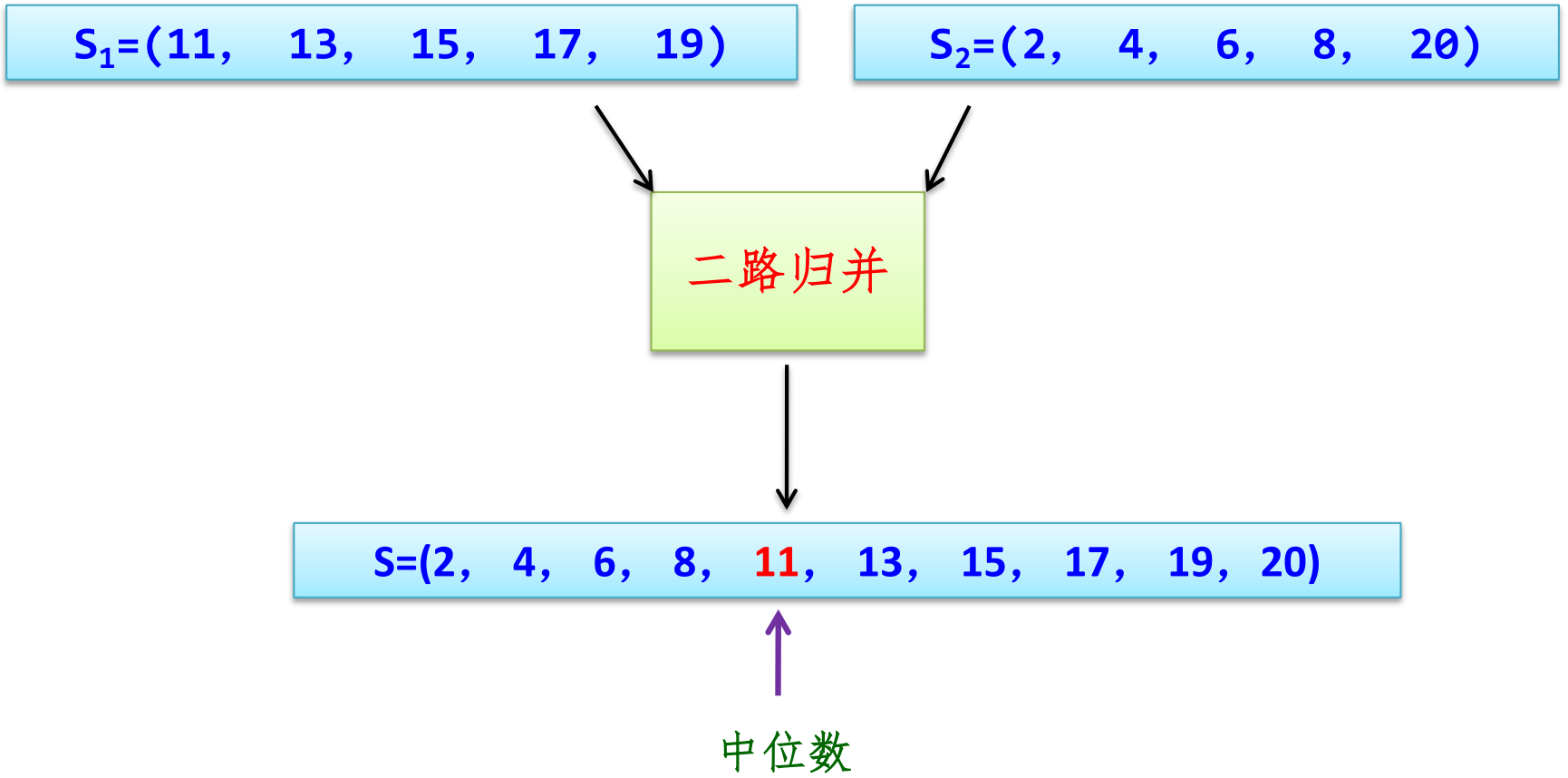
现有两个等长的升序序列A和B，试设计一个在时间和空间两方面都尽可能高效的算法，找出两个序列A和B的中位数。要求：

（1）给出算法的基本设计思想。

（2）根据设计思想，采用C、C++或Java语言描述算法，关键之处给出注释。

（3）说明你所设计算法的时间复杂度和空间复杂度。

思路



实际上，不需要求出S的全部元素，用*k*记录当前归并的元素个数，当*k*=*n*时，归并的那个元素就是中位数。

```
template <typename T>
T Middle(SqList<T> A,SqList<T> B)      //求解算法
{  int i=0,j=0;                        //i,j分别遍历A和B
   int k=0;                            //累计归并的次数
   while (i<A.length && j<B.length)    //两个有序顺序表均没有遍历完
   {  k++;                             //归并次数增1
      if (A.data[i]<B.data[j])         //A中当前元素为较小的元素
      {  if (k==A.length)              //恰好归并了n次
            return A.data[i];          //返回A中的当前元素
         i++;
      }
      else                             //B中当前元素为较小的元素
      {  if (k==B.length)              //恰好归并了n次
            return B.data[j];          //返回B中的当前元素
         j++;
      }
   }
}
```

算法的时间复杂度为O(*n*)，空间复杂度为O(1)。